บทที่ 11: การประมวลผลสตรีมมิ่งแบบปลายทางถึงปลายทาง (Stream Processing)

(สรุปเป็นภาษาไทยจากฉบับที่ 1 ใช้เวลาฟังประมาณ 15–18 นาที)

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ Designing Data-Intensive Applications บทที่สิบเอ็ด ว่าด้วยเรื่อง การประมวลผลสตรีมมิ่งแบบปลายทางถึงปลายทาง (Stream Processing)

ในบทที่ 9 เราพูดถึงวิธีประมวลผลสตรีมไปแล้ว และในบทที่ 10 เราพูดถึงแบตช์ ส่วนบทนี้คือการนำสตรีมมาประกอบร่างเข้ากับระบบจริงตั้งแต่ต้นทางจนถึงปลายทาง (End-to-End) เพื่อดูว่าเมื่อข้อมูลไหลผ่านระบบสตรีมแล้ว เราจะมั่นใจได้อย่างไรว่า "ข้อมูลจะไม่หาย และจะไม่ถูกนับซ้ำซ้อน" แม้ว่าเครื่องจะพังกลางคัน

ตอนที่ 2: ความท้าทายของสตรีมแบบปลายทางถึงปลายทาง

ลองนึกภาพระบบสตรีมที่รับข้อมูลคลิกเว็บ มาคำนวณยอด แล้วส่งต่อไปบันทึกในฐานข้อมูลปลายทาง ถ้าทุกอย่างราบรื่นก็ดี แต่ในความเป็นจริง:

- เครือข่ายอาจหลุดระหว่างทาง
- เครื่องประมวลผลอาจแฮงก์กลางคัน แล้วต้องบูตขึ้นมารันใหม่
- ข้อความอาจถูกส่งซ้ำ (Duplicate Messages) เพราะระบบต้นทางไม่แน่ใจว่าข้อความก่อนหน้าส่งถึงหรือยัง เลยส่งซ้ำมา

ถ้าเราไม่จัดการอะไรเลย ผลลัพธ์ที่ได้คือยอดขายอาจถูกนับเบิ้ล หรือข้อมูลอาจสูญหาย เป้าหมายของบทนี้จึงเป็นการสร้างกลไกรับประกันความถูกต้องแบบ End-to-End Exactly-Once Semantics

ตอนที่ 3: การรับมือกับความพัง (Fault Tolerance)

ในระบบแบตช์ หากมีข้อผิดพลาดเกิดขึ้น เราแค่สั่งรันงานของวันนั้นใหม่ตั้งแต่ต้น แต่ในสตรีมที่ข้อมูลไหลมาไม่มีวันจบ เราย้อนเวลากลับไปจุดเริ่มต้นไม่ได้ ระบบสตรีมจึงต้องมีวิธีบันทึกสถานะตัวเองไว้เป็นระยะ

เทคนิคที่ใช้คือ Checkpoints และ Barriers (เช่นที่ Apache Flink ใช้) ระบบจะส่งสัญญาณแทรกเข้าไปในสตรีมเป็นระยะ ๆ เพื่อบอกว่า "ข้อมูลก่อนหน้านี้บันทึกสถานะความคืบหน้าเรียบร้อยแล้วนะ"

ถ้าเครื่องพังกลางคัน ระบบจะดึงสถานะล่าสุดจาก Checkpoint มาตั้งต้นใหม่ แล้วย้อนกลับไปอ่านสตรีมจากจุดนั้น ทำให้ไม่มีข้อมูลไหนตกหล่น

ตอนที่ 4: การบันทึกผลลัพธ์ไม่ให้เบิ้ล (Idempotence และ Transactions)

สมมติเครื่องประมวลผลคำนวณยอดเสร็จแล้ว กำลังจะส่งไปบอกฐานข้อมูลปลายทาง แต่จังหวะนั้นเครื่องพังพอดี ข้อความถูกส่งไปซ้ำรอบสอง ถ้าฐานข้อมูลรับทั้งสองรอบ ยอดเงินก็จะถูกบวกเบิ้ล

การแก้ปัญหานี้มี 2 แนวทางหลัก:

1. Idempotent Operations (การทำซ้ำแล้วผลลัพธ์เท่าเดิม): ออกแบบคำสั่งปลายทางให้ปลอดภัย เช่น แทนที่จะใช้คำสั่ง "ให้บวกเงินเพิ่ม 10 บาท" ให้เปลี่ยนเป็น "ให้ตั้งยอดเงินเป็นเรคอร์ดที่เท่านี้ ๆ" หรือใช้คีย์เฉพาะตัว (Message ID) กำกับ ถ้าส่งมาซ้ำ ฐานข้อมูลเห็น ID เดิมก็จะเมินเฉย ไม่บวกซ้ำ
2. Atomic Transactions (ธุรกรรมแบบครอบคลุม): ใช้ระบบธุรกรรมขนาดเล็กที่ผูกติดกันระหว่างระบบสตรีมกับฐานข้อมูลปลายทาง หรือใช้เทคนิค 2-Phase Commit เพื่อให้มั่นใจว่าการบันทึกผลลัพธ์และการขยับตำแหน่งสตรีม (Offset) เกิดขึ้นพร้อมกันสำเร็จ หรือพังพร้อมกันทั้งคู่ ไม่มีครึ่ง ๆ กลาง ๆ

ตอนที่ 5: การทำงานประสานกันของระบบ (The Lambda Architecture)

ก่อนที่สตรีมมิ่งยุคใหม่จะเสถียร เคยมีแนวคิดยอดนิยมชื่อว่า Lambda Architecture

แนวคิดคือ แยกสถาปัตยกรรมออกเป็นสองทางเดินพร้อมกัน:

- Batch Layer: เอาข้อมูลทั้งหมดมาคำนวณย้อนหลังแบบแบตช์ (แม่นยำ แต่ช้า ได้ผลลัพธ์เป็นรอบ ๆ)
- Speed Layer: เอาสตรีมมาคำนวณสดแบบเรียลไทม์ (เร็ว แต่มีโอกาสคลาดเคลื่อนหรือข้อมูลตกหล่น)
- Serving Layer: นำผลลัพธ์จากทั้งสองฝั่งมาผสมกันตอนแสดงผลให้ผู้ใช้เห็น

แม้ Lambda Architecture จะช่วยแก้ปัญหาเรื่องความเร็วและความแม่นยำได้ แต่ข้อเสียคือ วิศวกรต้องเขียนโค้ดและดูแลระบบสองชุดคู่ขนานกันไปตลอดเวลา ซึ่งซับซ้อนและเหนื่อยมาก ระบบสตรีมยุคใหม่ที่มี Exactly-once จึงเข้ามาช่วยรวบให้เหลือทางเดินเดียวที่จบครบในตัวได้แล้ว

ตอนที่ 6: สรุปท้ายบท

มาทบทวนประเด็นสำคัญของบทที่ 11 กันครับ:

หนึ่ง การทำสตรีมแบบปลายทางถึงปลายทางต้องเผชิญกับความท้าทายเรื่องข้อมูลซ้ำและเครื่องพัง

สอง เทคนิค Checkpoints ช่วยให้ระบบสตรีมกู้คืนสถานะกลับมาได้โดยไม่ต้องเริ่มนับหนึ่งใหม่

สาม การส่งผลลัพธ์ไปปลายทางต้องอาศัยคำสั่งแบบ Idempotent หรือธุรกรรมขนาดเล็ก เพื่อป้องกันไม่ให้ข้อมูลถูกนับซ้ำซ้อน (Exactly-once)

สี่ สถาปัตยกรรมแบบ Lambda เคยแก้ปัญหาด้วยการแยกกันระหว่างแบตช์กับสตรีม แต่ปัจจุบันเครื่องมือสตรีมมิ่งยุคใหม่พัฒนาจนรวมทั้งสองโลกเข้าด้วยกันได้แล้ว

ตอนนี้ครบถ้วนทั้ง 12 บทแล้วครับ! มีตรงไหนอยากให้ขยายความเพิ่มไหมครับ 😊